DevOps Monitoring Setup Guide

Prometheus + Grafana on AWS EC2

Step-by-Step Installation, Integration & Dashboard Guide

Muhammad Wahaj Naveed
fa22-BSE-100

EC2 IP	`100.53.31.202`
Jenkins	`http://100.53.31.202:8080/`
Web App	`http://100.53.31.202:3000/`
Prometheus	`http://100.53.31.202:9090/`
Grafana	`http://100.53.31.202:3030/`

ARCHITECTURE DECISION: Both Grafana and Prometheus will be installed on your EXISTING EC2 instance (same server as Jenkins). No new instance is needed for a basic monitoring setup. Grafana will run on port 3030 (since 3000 is used by your web app).

Architecture Overview

Before starting, understand what you are building:

Your App	Prometheus	Grafana	Node Exporter
:3000	:9090	:3030	:9100

How it works:

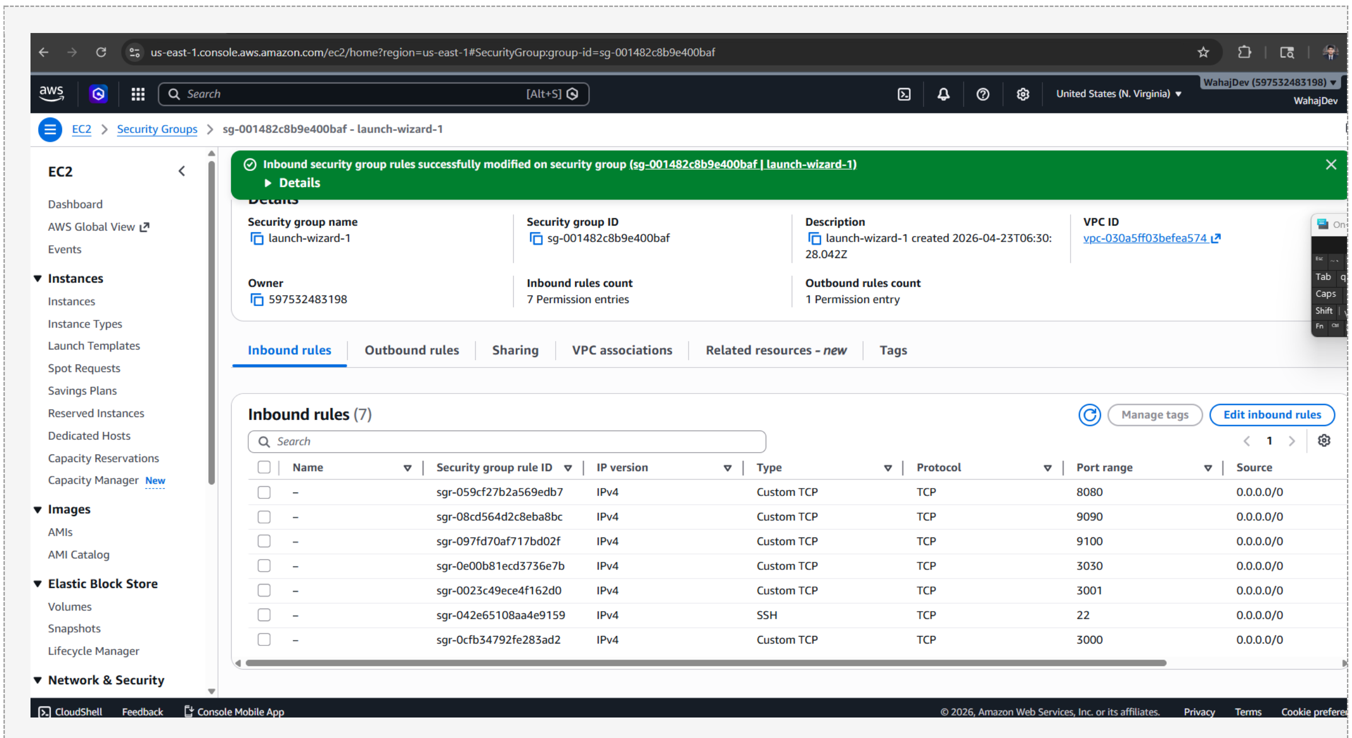
- Node Exporter runs on your EC2 and exposes system metrics (CPU, RAM, Disk) on port 9100
- Prometheus scrapes (collects) those metrics every 15 seconds and stores them
- Grafana connects to Prometheus as a data source and displays beautiful dashboards
- Jenkins metrics can also be scraped by Prometheus using a plugin

Pre-Requisites: Open EC2 Security Group Ports

BEFORE anything else, you MUST open these ports in your AWS EC2 Security Group. Without this, you cannot access the tools from your browser.

> Go to AWS Console > EC2 > Your Instance > Security > Security Groups > Edit Inbound Rules

Port	Protocol	Source	Purpose
9090	TCP	0.0.0.0/0	Prometheus Web UI
9100	TCP	0.0.0.0/0	Node Exporter metrics
3030	TCP	0.0.0.0/0	Grafana Web UI



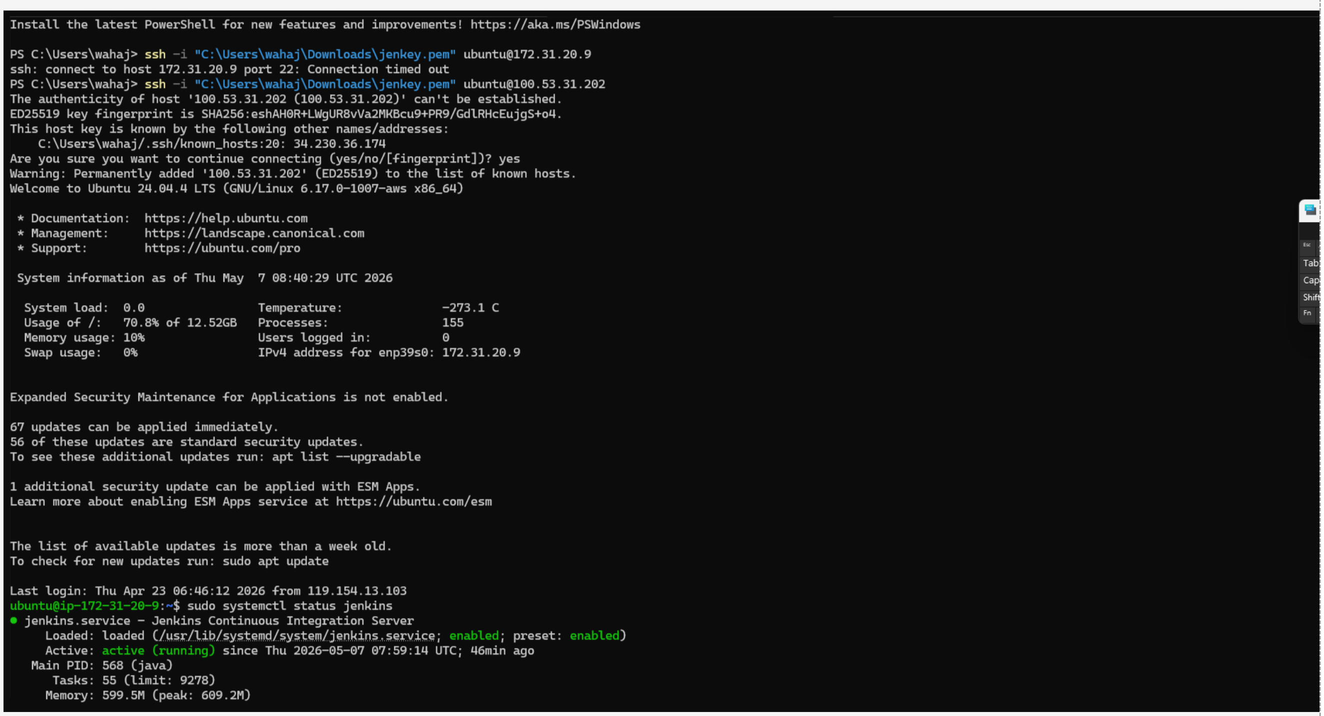
Part 1: Install Prometheus

Prometheus is a time-series database that collects and stores metrics. It will scrape your EC2 server and Jenkins.

Step 1.1 - Connect to Your EC2 via PowerShell

Open PowerShell on your Windows machine and connect:

```
# Replace with YOUR .pem key file path and EC2 public IP
ssh -i C:\path\to\your-key.pem ubuntu@100.53.31.202

# Once connected, you should see a prompt like:
# ubuntu@ip-100-53-31-202:~$
```



Step 1.2 - Create Prometheus System User

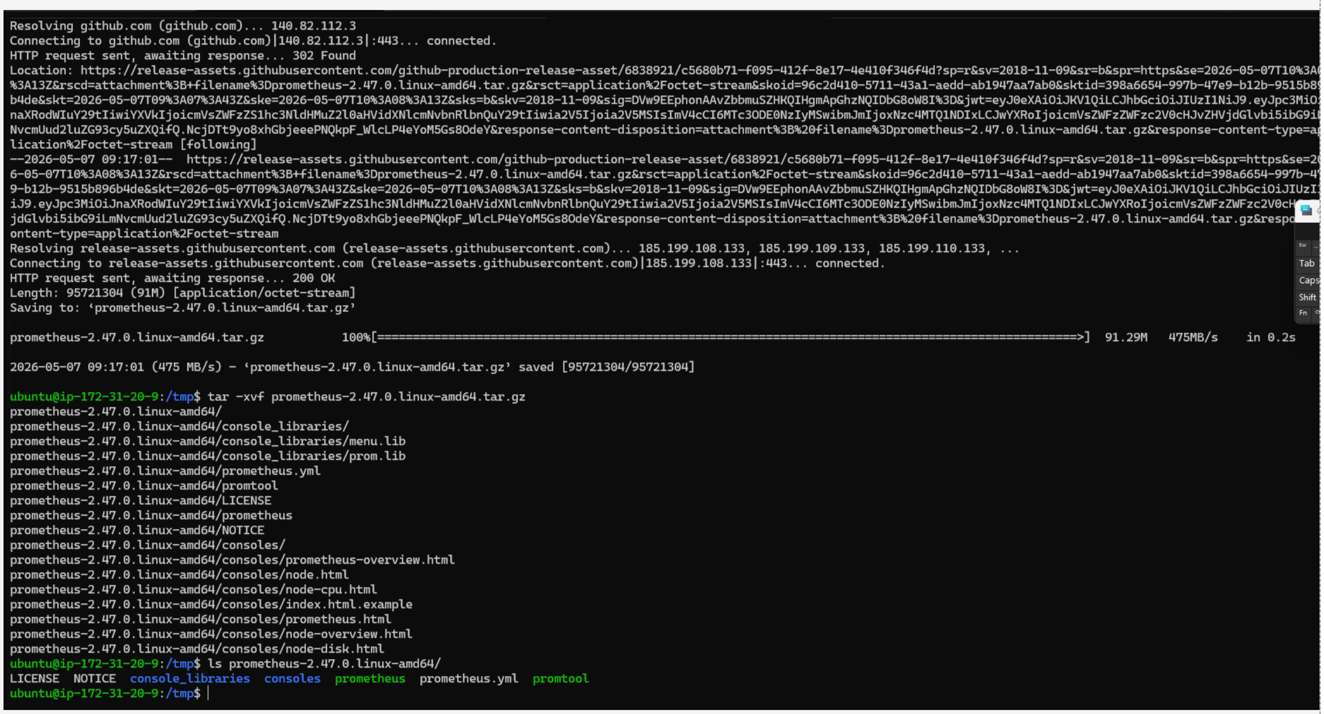
We create a dedicated system user for Prometheus (best security practice):

```
sudo useradd --no-create-home --shell /bin/false prometheus
```

Step 1.3 - Download Prometheus

Download the latest Prometheus binary:

```
cd /tmp

# Download Prometheus 2.47.0 (stable version)
wget https://github.com/prometheus/prometheus/releases/download/v2.47.0/prometheus-2.47.0.linux-amd64.tar.gz

# Extract the archive
tar -xvf prometheus-2.47.0.linux-amd64.tar.gz

# List to confirm extraction
ls prometheus-2.47.0.linux-amd64/
```



Step 1.4 - Move Binaries and Create Directories

```
# Create required directories
sudo mkdir /etc/prometheus
sudo mkdir /var/lib/prometheus

# Move the main binaries
sudo cp /tmp/prometheus-2.47.0.linux-amd64/prometheus /usr/local/bin/
sudo cp /tmp/prometheus-2.47.0.linux-amd64/promtool /usr/local/bin/

# Move console files
sudo cp -r /tmp/prometheus-2.47.0.linux-amd64/consoles /etc/prometheus
sudo cp -r /tmp/prometheus-2.47.0.linux-amd64/console_libraries /etc/prometheus

# Set ownership to prometheus user
sudo chown -R prometheus:prometheus /etc/prometheus
sudo chown -R prometheus:prometheus /var/lib/prometheus
```

```
sudo chown prometheus:prometheus /usr/local/bin/prometheus
sudo chown prometheus:prometheus /usr/local/bin/promtool
```

Step 1.5 - Create Prometheus Configuration File

This is the main config file. It tells Prometheus WHAT to scrape and HOW often:

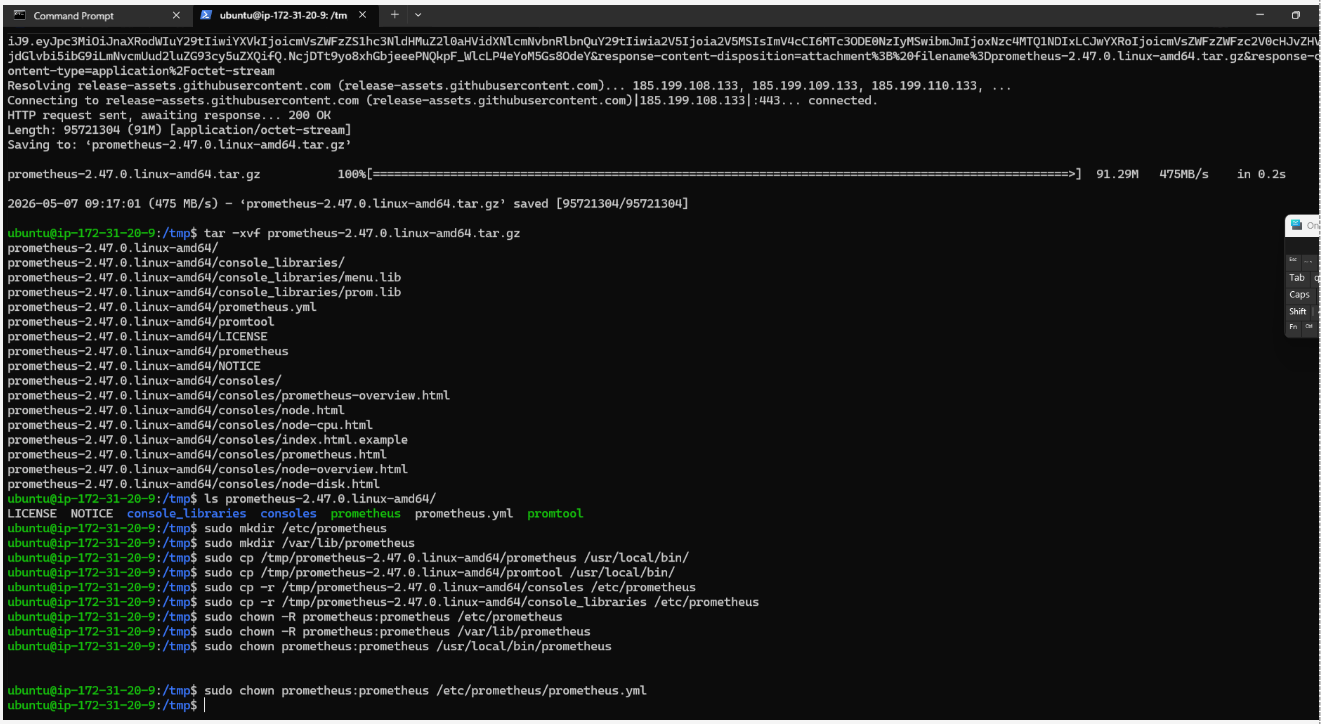
```
sudo nano /etc/prometheus/prometheus.yml
```

Paste the following content exactly (use Ctrl+Shift+V to paste in nano):

```
global:
  scrape_interval: 15s       # Scrape every 15 seconds
  evaluation_interval: 15s   # Evaluate rules every 15 seconds

scrape_configs:

  # Prometheus scrapes itself
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']

  # Node Exporter - system metrics (CPU, RAM, Disk)
  - job_name: 'node_exporter'
    static_configs:
      - targets: ['localhost:9100']

  # Jenkins monitoring
  - job_name: 'jenkins'
    metrics_path: '/prometheus'
    static_configs:
      - targets: ['localhost:8080']
```

Save the file: press Ctrl+X, then Y, then Enter

```
# Set ownership of the config file
sudo chown prometheus:prometheus /etc/prometheus/prometheus.yml
```



Step 1.6 - Create Prometheus Systemd Service

This makes Prometheus run automatically and restart if it crashes:

```
sudo nano /etc/systemd/system/prometheus.service
```

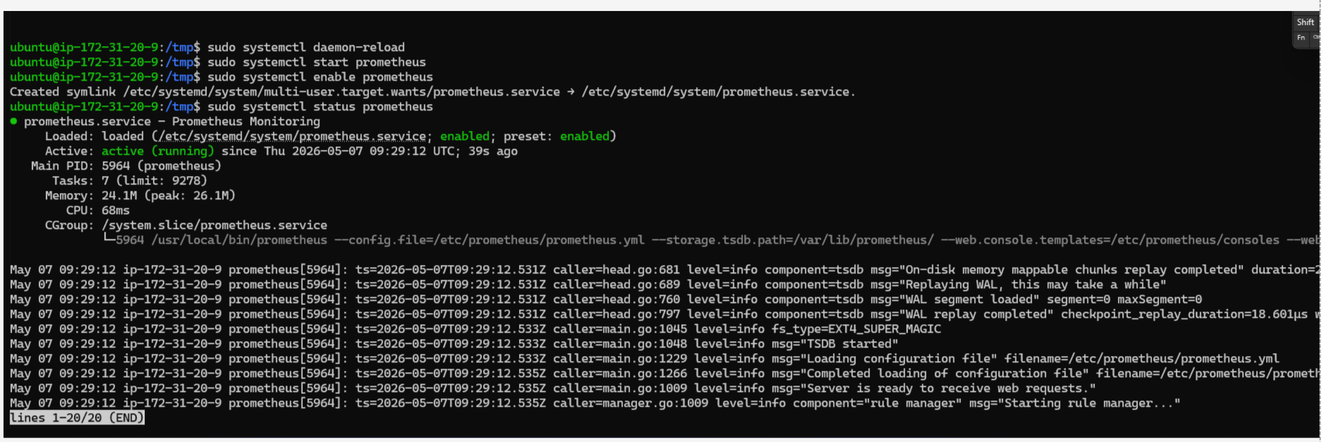
Paste this content:

```
[Unit]
Description=Prometheus Monitoring
Wants=network-online.target
After=network-online.target

[Service]
User=prometheus
Group=prometheus
Type=simple
ExecStart=/usr/local/bin/prometheus \
    --config.file=/etc/prometheus/prometheus.yml \
    --storage.tsdb.path=/var/lib/prometheus/ \
    --web.console.templates=/etc/prometheus/consoles \
    --web.console.libraries=/etc/prometheus/console_libraries \
    --web.listen-address=0.0.0.0:9090

[Install]
WantedBy=multi-user.target
```

Save: Ctrl+X, then Y, then Enter

Step 1.7 - Start and Enable Prometheus

```
# Reload systemd to recognize new service
sudo systemctl daemon-reload

# Start Prometheus
sudo systemctl start prometheus

# Enable it to start on server reboot
sudo systemctl enable prometheus

# Check status - should show 'active (running)'
sudo systemctl status prometheus
```

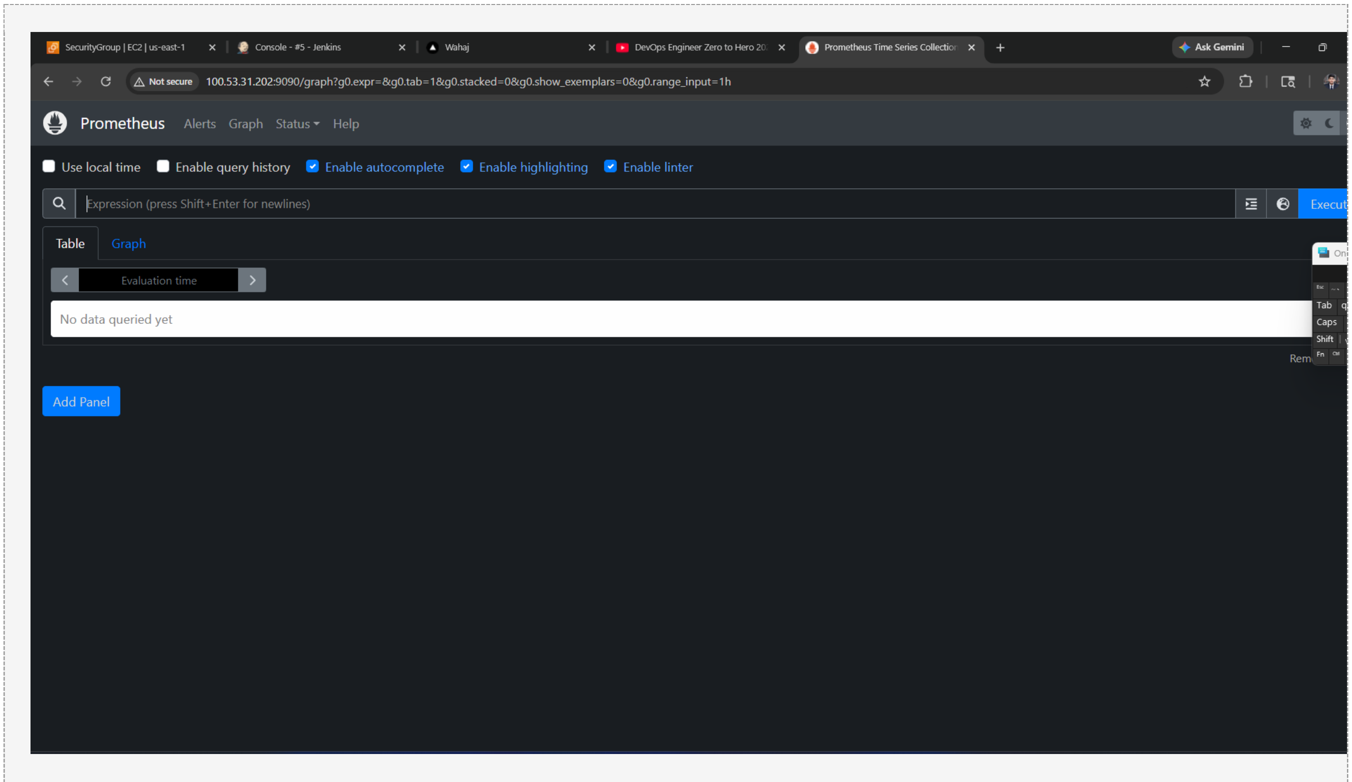


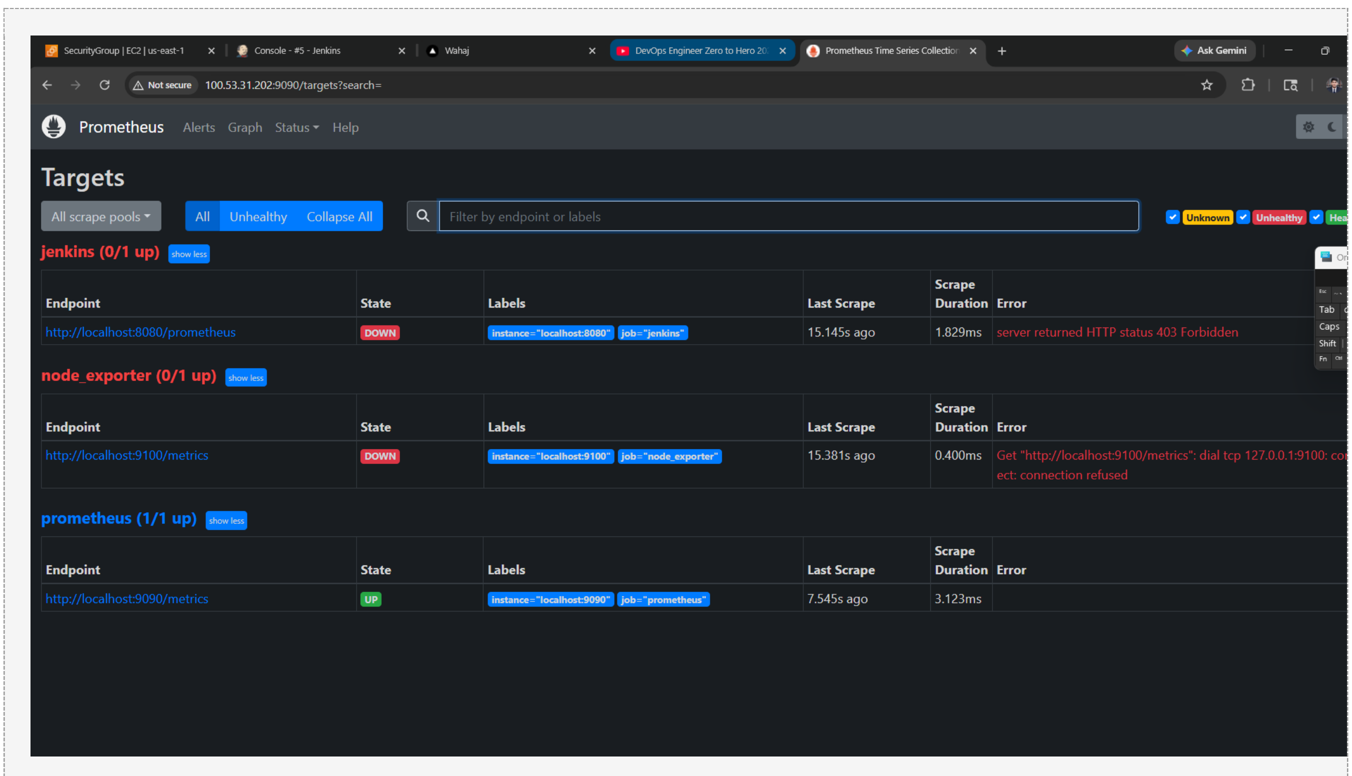
Step 1.8 - Verify Prometheus is Running

Open your browser and go to:

```
http://100.53.31.202:9090
```

You should see the Prometheus web interface. Click on Status > Targets to verify scrape targets.


SecurityGroup | EC2 | us-east-1
Console - #5 - Jenkins
Wahaj
Prometheus Time Series Collection
Ask Gemini
Not secure
100.53.31.202:9090/graph?g0.expr=&g0.tab=1&g0.stacked=0&g0.show_exemplars=0&g0.range_input=1h
Prometheus
Alerts
Graph
Status
Help
Use local time
Enable query history
Enable autocomplete
Enable highlighting
Enable linter
Expression (press Shift+Enter for newlines)
Table
Graph
Evaluation time
No data queried yet
Add Panel


SecurityGroup | EC2 | us-east-1
Console - #5 - Jenkins
Wahaj
Prometheus Time Series Collection
Ask Gemini
Not secure
100.53.31.202:9090/targets?search=
Prometheus
Alerts
Graph
Status
Help
Targets
All scrape pools
All
Unhealthy
Collapse All
Filter by endpoint or labels
Unknown
Unhealthy
jenkins (0/1 up)
show less
Endpoint
State
Labels
Last Scrape
Scrape Duration
Error
http://localhost:8080/prometheus
DOWN
instance="localhost:8080"
job="jenkins"
15.145s ago
1.829ms
server returned HTTP status 403 Forbidden
node_exporter (0/1 up)
show less
http://localhost:9100/metrics
DOWN
instance="localhost:9100"
job="node_exporter"
15.381s ago
0.400ms
Get "http://localhost:9100/metrics": dial tcp 127.0.0.1:9100:
ect: connection refused
prometheus (1/1 up)
show less
http://localhost:9090/metrics
UP
instance="localhost:9090"
job="prometheus"
7.545s ago
3.123ms

Part 2: Install Node Exporter

Node Exporter is a small agent that exposes your EC2 server's system metrics (CPU, Memory, Disk, Network) to Prometheus.

Step 2.1 - Download and Install Node Exporter

```
# Create system user for Node Exporter
sudo useradd --no-create-home --shell /bin/false node_exporter

# Download Node Exporter
cd /tmp
wget
https://github.com/prometheus/node_exporter/releases/download/v1.6.1/node_exporter-1.6.1.linux-amd64.tar.gz

# Extract
tar -xvf node_exporter-1.6.1.linux-amd64.tar.gz

# Move binary
sudo cp /tmp/node_exporter-1.6.1.linux-amd64/node_exporter /usr/local/bin/

# Set ownership
sudo chown node_exporter:node_exporter /usr/local/bin/node_exporter
```

Step 2.2 - Create Node Exporter Service

```
sudo nano /etc/systemd/system/node_exporter.service
```

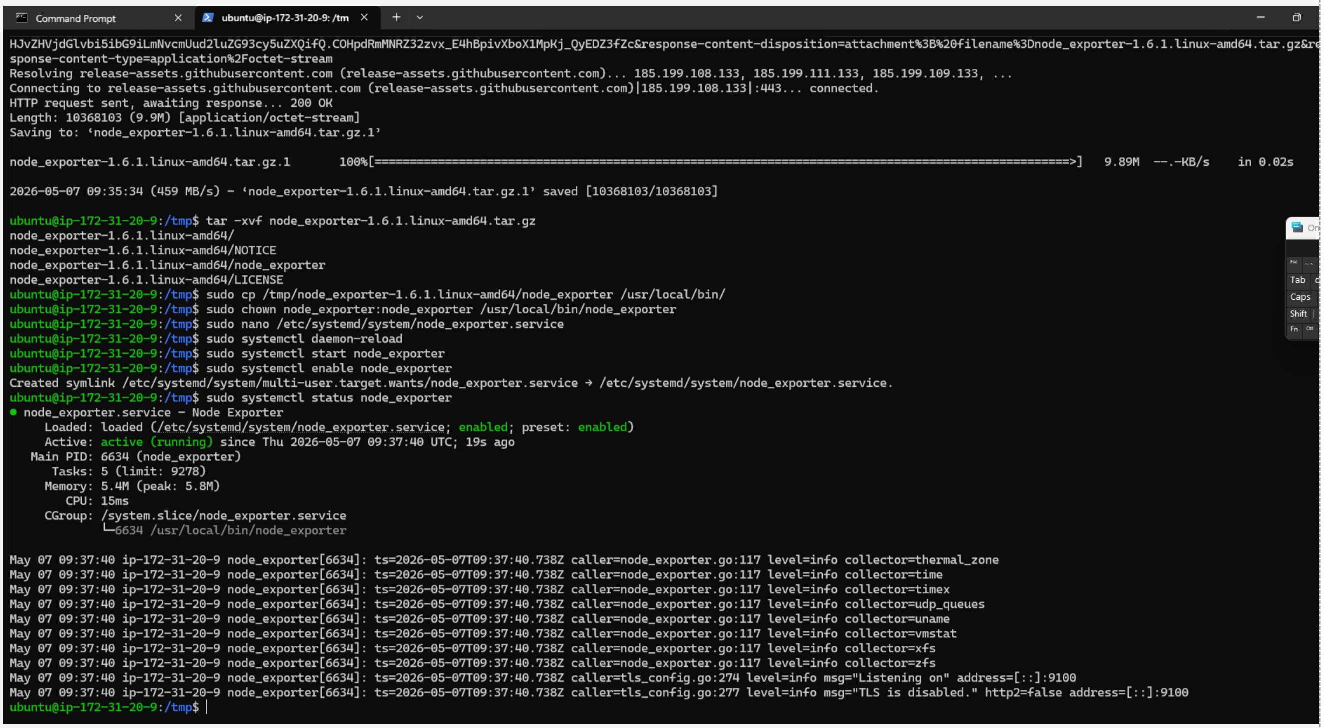
Paste this content:

```
[Unit]
Description=Node Exporter
Wants=network-online.target
After=network-online.target

[Service]
User=node_exporter
Group=node_exporter
Type=simple
ExecStart=/usr/local/bin/node_exporter

[Install]
WantedBy=multi-user.target
```

Step 2.3 - Start Node Exporter

```
sudo systemctl daemon-reload
sudo systemctl start node_exporter
```

```
sudo systemctl enable node_exporter
sudo systemctl status node_exporter
```

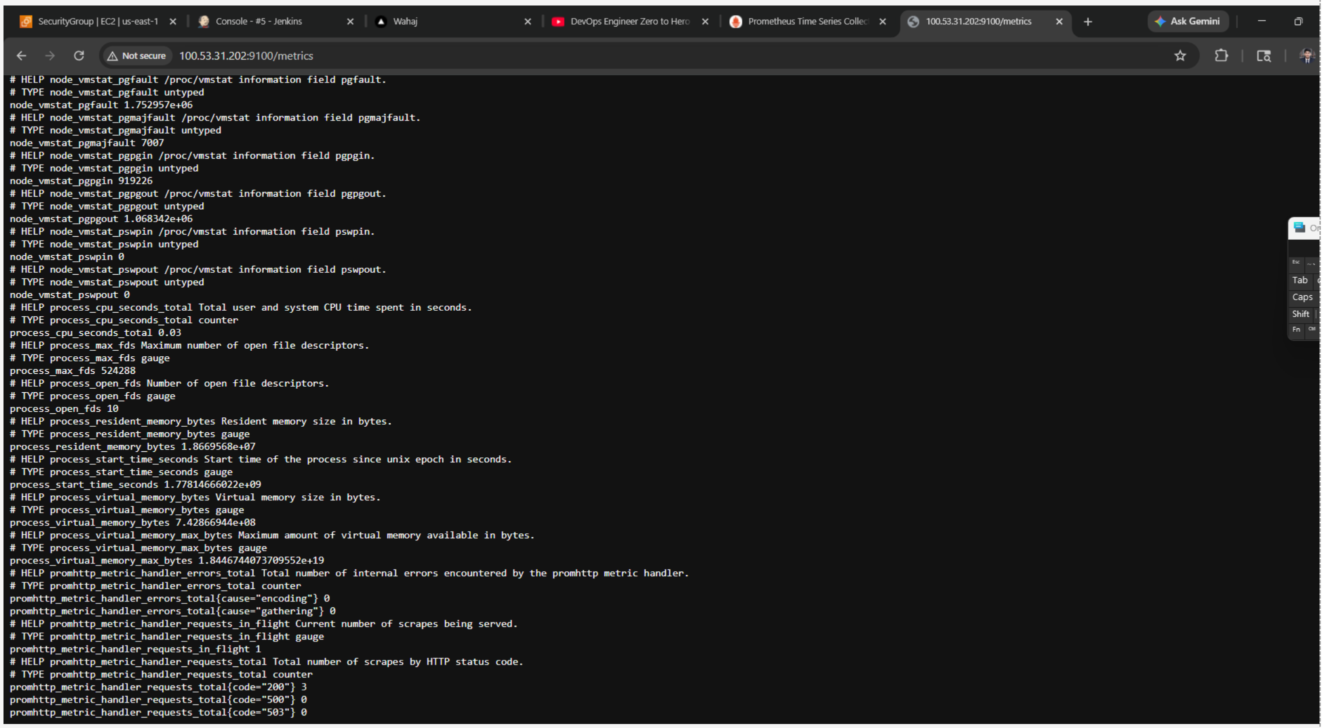


Step 2.4 - Verify Node Exporter Metrics

Open browser and visit:

```
http://100.53.31.202:9100/metrics
```

You should see a long list of system metrics in plain text. This is what Prometheus will scrape.

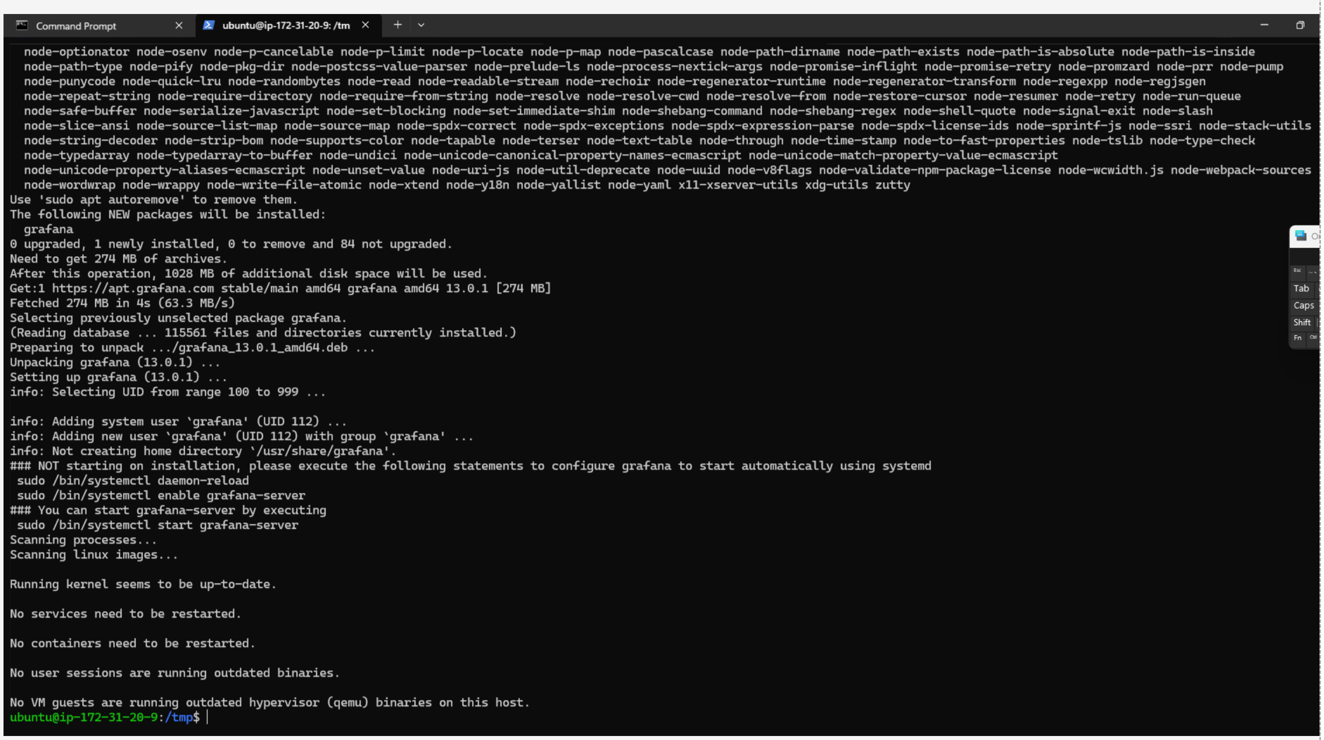

SecurityGroup | EC2 | us-east-1
Console - #5 - Jenkins
Wahaj
DevOps Engineer Zero to Hero
Prometheus Time Series Collec
100.53.31.202:9100/metrics
Ask Gemini
Not secure
100.53.31.202:9100/metrics
HELP node_vmstat_pgfault /proc/vmstat information field pgfault.
TYPE node_vmstat_pgfault untyped
node_vmstat_pgfault 1.752957e+06
HELP node_vmstat_pgmajfault /proc/vmstat information field pgmajfault.
TYPE node_vmstat_pgmajfault untyped
node_vmstat_pgmajfault 7007
HELP node_vmstat_pgpgin /proc/vmstat information field pgpgin.
TYPE node_vmstat_pgpgin untyped
node_vmstat_pgpgin 919226
HELP node_vmstat_pgpgout /proc/vmstat information field pgpgout.
TYPE node_vmstat_pgpgout untyped
node_vmstat_pgpgout 1.068342e+06
HELP node_vmstat_pswpin /proc/vmstat information field pswpin.
TYPE node_vmstat_pswpin untyped
node_vmstat_pswpin 0
HELP node_vmstat_pswpout /proc/vmstat information field pswpout.
TYPE node_vmstat_pswpout untyped
node_vmstat_pswpout 0
HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.
TYPE process_cpu_seconds_total counter
process_cpu_seconds_total 0.03
HELP process_max_fds Maximum number of open file descriptors.
TYPE process_max_fds gauge
process_max_fds 524288
HELP process_open_fds Number of open file descriptors.
TYPE process_open_fds gauge
process_open_fds 10
HELP process_resident_memory_bytes Resident memory size in bytes.
TYPE process_resident_memory_bytes gauge
process_resident_memory_bytes 1.8669568e+07
HELP process_start_time_seconds Start time of the process since unix epoch in seconds.
TYPE process_start_time_seconds gauge
process_start_time_seconds 1.77814666022e+09
HELP process_virtual_memory_bytes Virtual memory size in bytes.
TYPE process_virtual_memory_bytes gauge
process_virtual_memory_bytes 7.42866944e+08
HELP process_virtual_memory_max_bytes Maximum amount of virtual memory available in bytes.
TYPE process_virtual_memory_max_bytes gauge
process_virtual_memory_max_bytes 1.8446744073709552e+19
HELP promhttp_metric_handler_errors_total Total number of internal errors encountered by the promhttp metric handler.
TYPE promhttp_metric_handler_errors_total counter
promhttp_metric_handler_errors_total{cause="encoding"} 0
promhttp_metric_handler_errors_total{cause="gathering"} 0
HELP promhttp_metric_handler_requests_in_flight Current number of scrapes being served.
TYPE promhttp_metric_handler_requests_in_flight gauge
promhttp_metric_handler_requests_in_flight 1
HELP promhttp_metric_handler_requests_total Total number of scrapes by HTTP status code.
TYPE promhttp_metric_handler_requests_total counter
promhttp_metric_handler_requests_total{code="200"} 3
promhttp_metric_handler_requests_total{code="500"} 0
promhttp_metric_handler_requests_total{code="503"} 0

Part 3: Install Grafana

Grafana is the visualization tool. It connects to Prometheus and creates beautiful dashboards. We install it on the SAME EC2 instance but on port 3030 (since 3000 is occupied by your Docker web app).

Step 3.1 - Add Grafana Repository and Install

```
# Install required dependencies
sudo apt-get install -y apt-transport-https software-properties-common wget

# Add Grafana GPG key
sudo mkdir -p /etc/apt/keyrings
wget -q -O - https://apt.grafana.com/gpg.key | gpg --dearmor | sudo tee
/etc/apt/keyrings/grafana.gpg > /dev/null

# Add Grafana repository
echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] https://apt.grafana.com stable
main" | sudo tee /etc/apt/sources.list.d/grafana.list

# Update package list
sudo apt-get update

# Install Grafana
sudo apt-get install -y grafana
```



Step 3.2 - Change Grafana Port to 3030

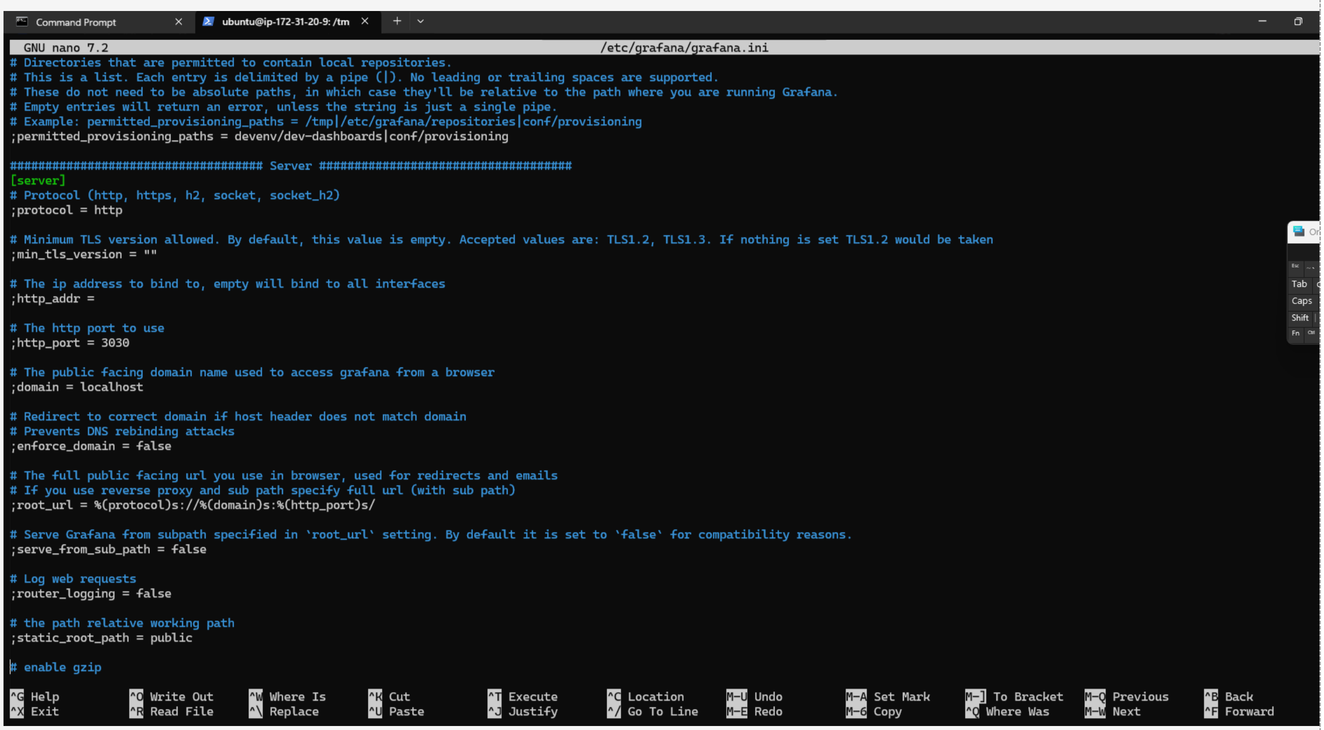
Since your web app is on port 3000, we change Grafana to use port 3030:

```
sudo nano /etc/grafana/grafana.ini
```

Find the [server] section (press Ctrl+W to search, type 'http_port'). Change the port:

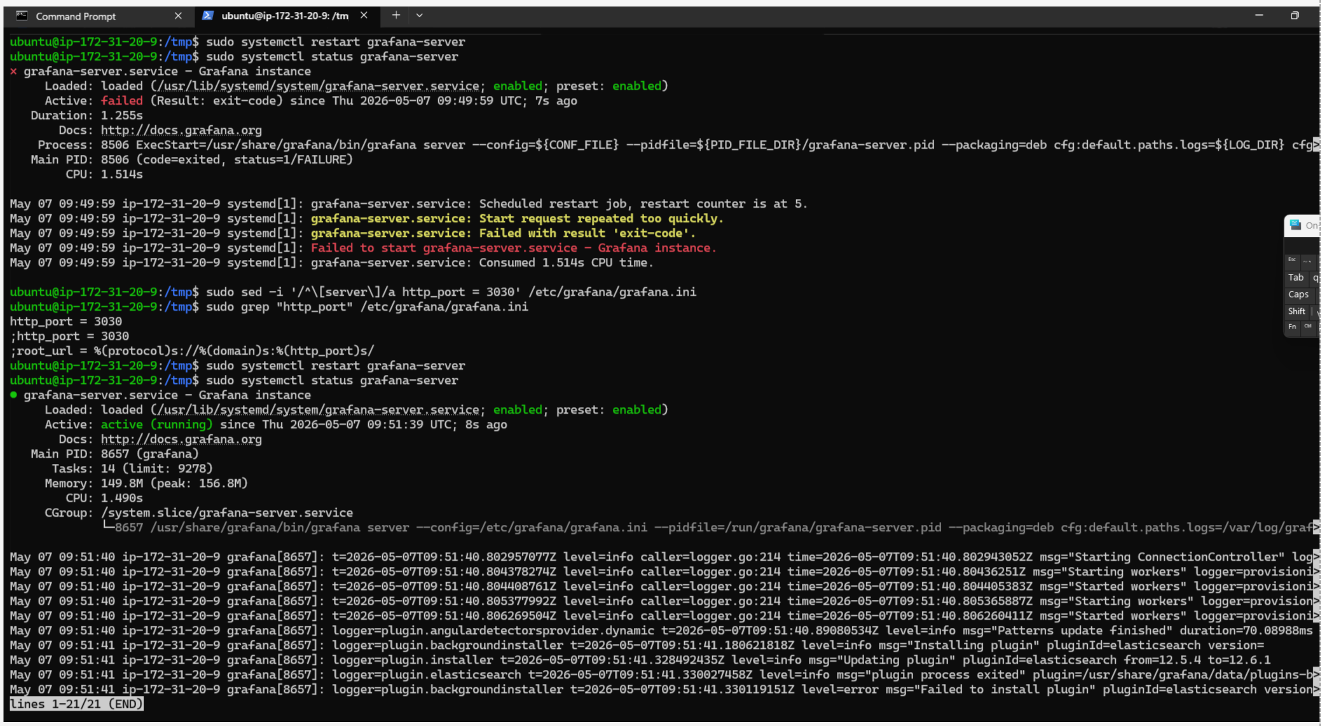
```
[server]
# The IP address to bind to, empty will bind to all interfaces
http_addr =

# The http port to use
http_port = 3030
```

Save: Ctrl+X, then Y, then Enter



Step 3.3 - Start and Enable Grafana

```
# Start Grafana service
sudo systemctl start grafana-server

# Enable on boot
sudo systemctl enable grafana-server

# Check status
sudo systemctl status grafana-server
```



Step 3.4 - Access Grafana Web UI

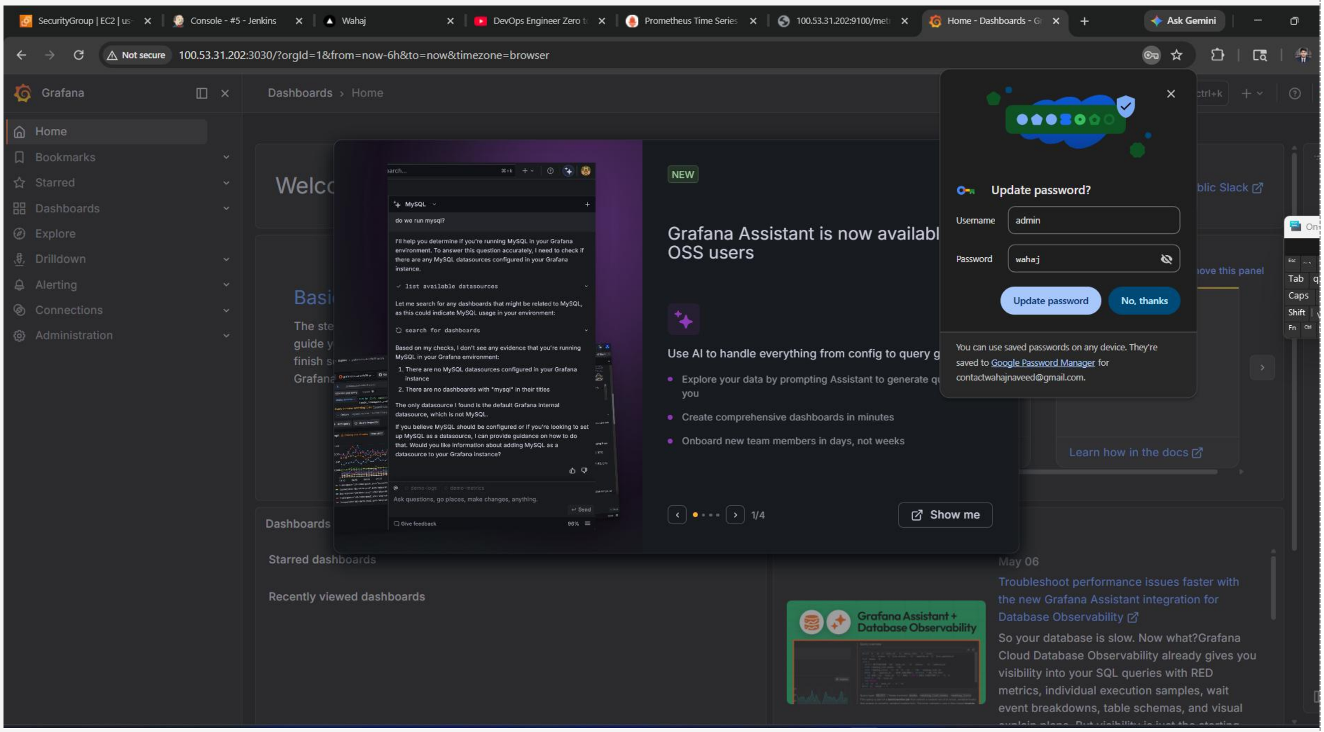
Open your browser and navigate to:

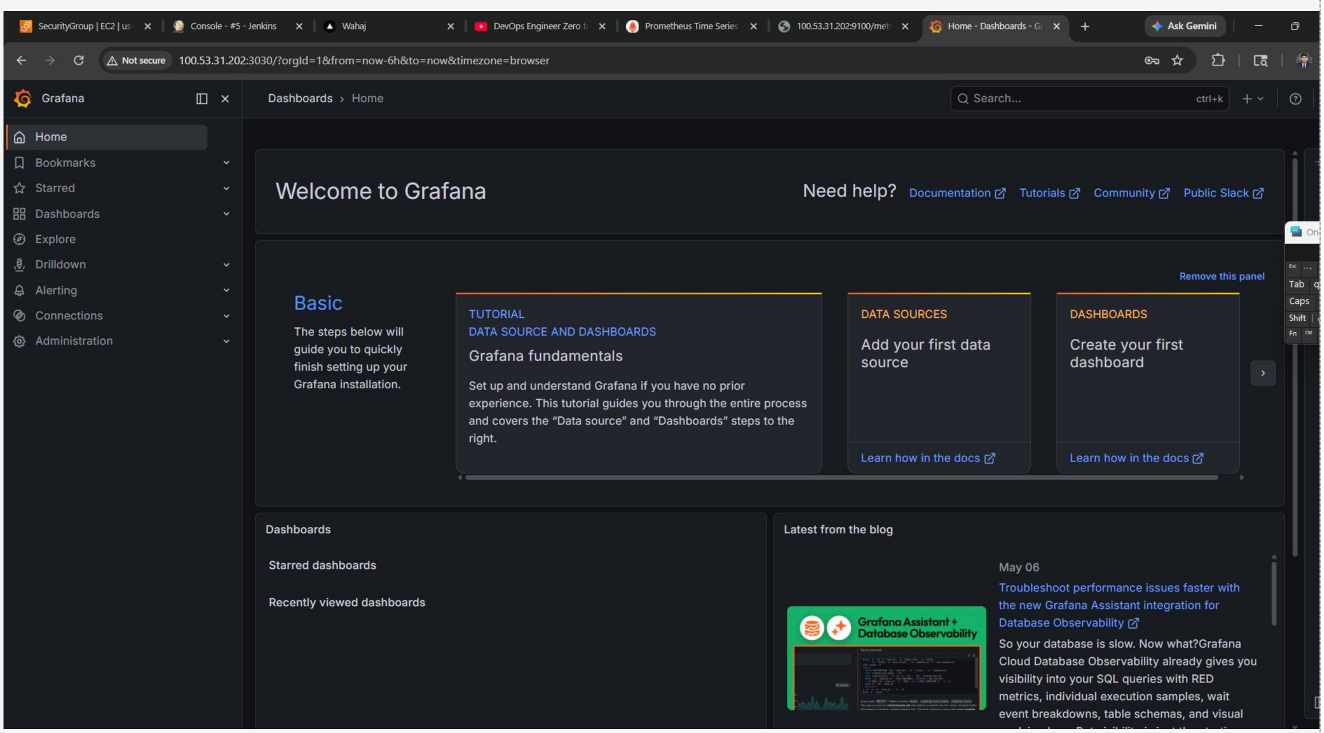
```
http://100.53.31.202:3030
```

You will see the Grafana login page. Use these default credentials:

Username	`admin`
Password	`admin`

> After first login, Grafana will ask you to change the password. Set a strong password and remember it!


100.53.31.202:3030/?orgId=1&from=now-6h&to=now&timezone=browser
Grafana
Home
Bookmarks
Starred
Dashboards
Explore
Drilldown
Alerting
Connections
Administration
Dashboards › Home
NEW
Grafana Assistant is now availabl
OSS users
Use AI to handle everything from config to query g
Explore your data by prompting Assistant to generate qu
you
Create comprehensive dashboards in minutes
Onboard new team members in days, not weeks
1/4
Show me
Update password?
Username
admin
Password
wahaj
Update password
No, thanks
You can use saved passwords on any device. They're saved to Google Password Manager for contactwahajnaveed@gmail.com.
Starred dashboards
Recently viewed dashboards
May 06
Troubleshoot performance issues faster with the new Grafana Assistant integration for Database Observability
So your database is slow. Now what?Grafana Cloud Database Observability already gives you visibility into your SQL queries with RED metrics, individual execution samples, wait event breakdowns, table schemas, and visual

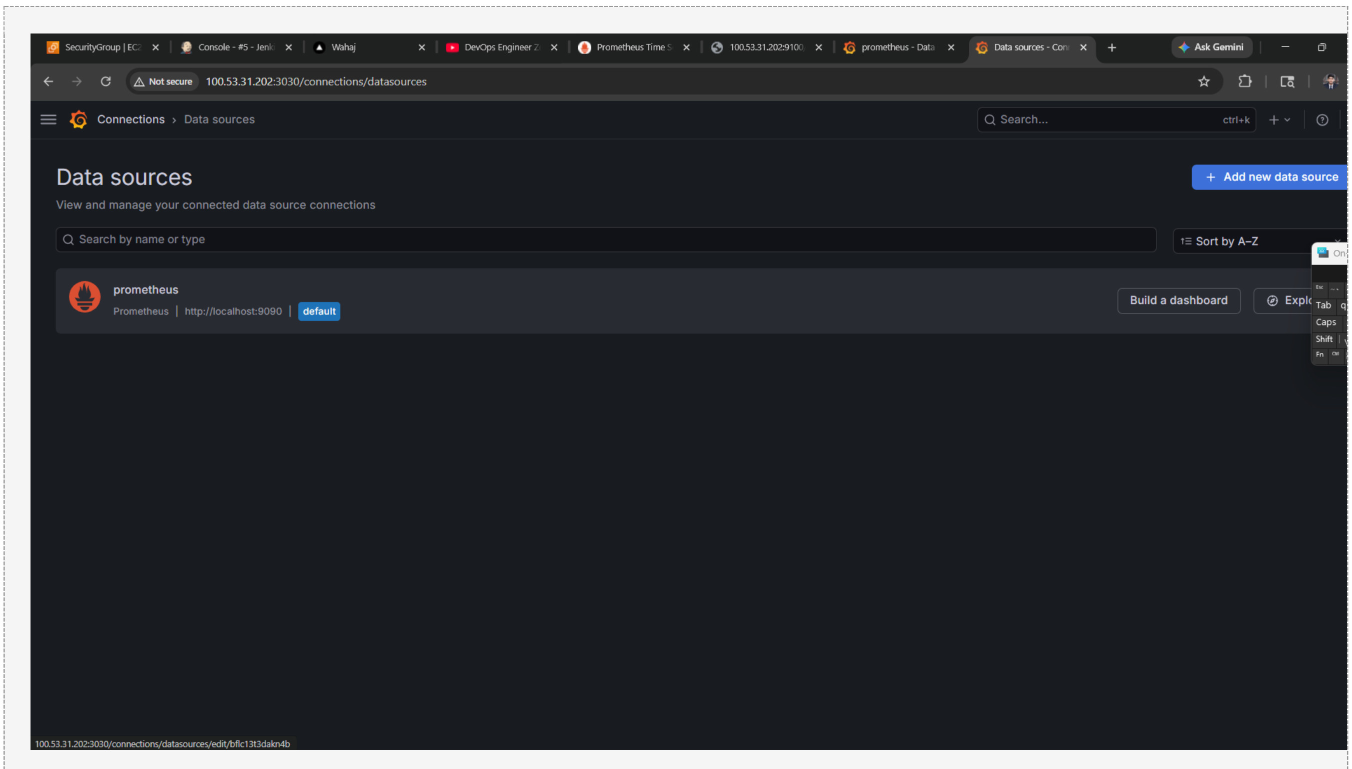

100.53.31.202:3030/?orgId=1&from=now-6h&to=now&timezone=browser
Grafana
Home
Bookmarks
Starred
Dashboards
Explore
Drilldown
Alerting
Connections
Administration
Dashboards › Home
Search...
Welcome to Grafana
Need help?
Documentation
Tutorials
Community
Public Slack
Remove this panel
Basic
The steps below will guide you to quickly finish setting up your Grafana installation.
TUTORIAL
DATA SOURCE AND DASHBOARDS
Grafana fundamentals
Set up and understand Grafana if you have no prior experience. This tutorial guides you through the entire process and covers the "Data source" and "Dashboards" steps to the right.
DATA SOURCES
Add your first data source
Learn how in the docs
DASHBOARDS
Create your first dashboard
Learn how in the docs
Dashboards
Starred dashboards
Recently viewed dashboards
Latest from the blog
May 06
Troubleshoot performance issues faster with the new Grafana Assistant integration for Database Observability
So your database is slow. Now what?Grafana Cloud Database Observability already gives you visibility into your SQL queries with RED metrics, individual execution samples, wait event breakdowns, table schemas, and visual

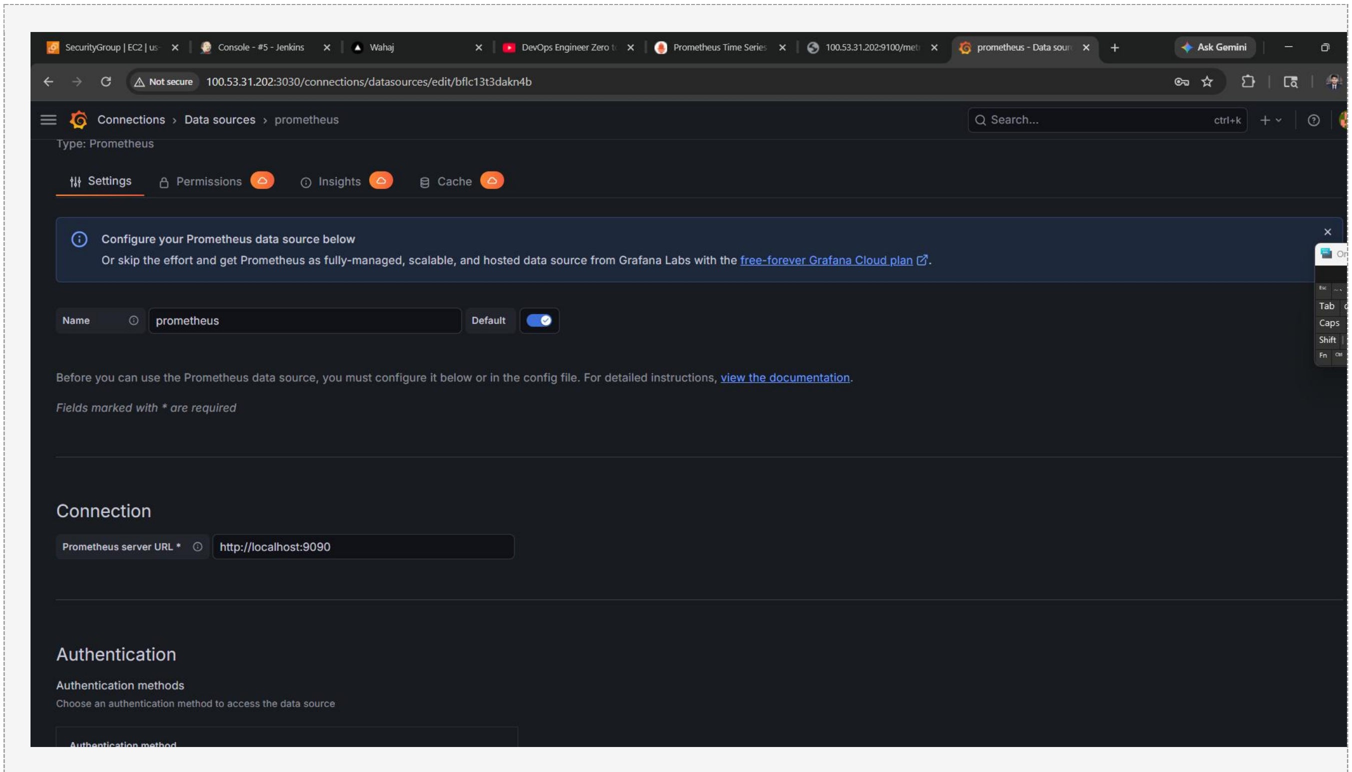
Part 4: Integrate Grafana with Prometheus

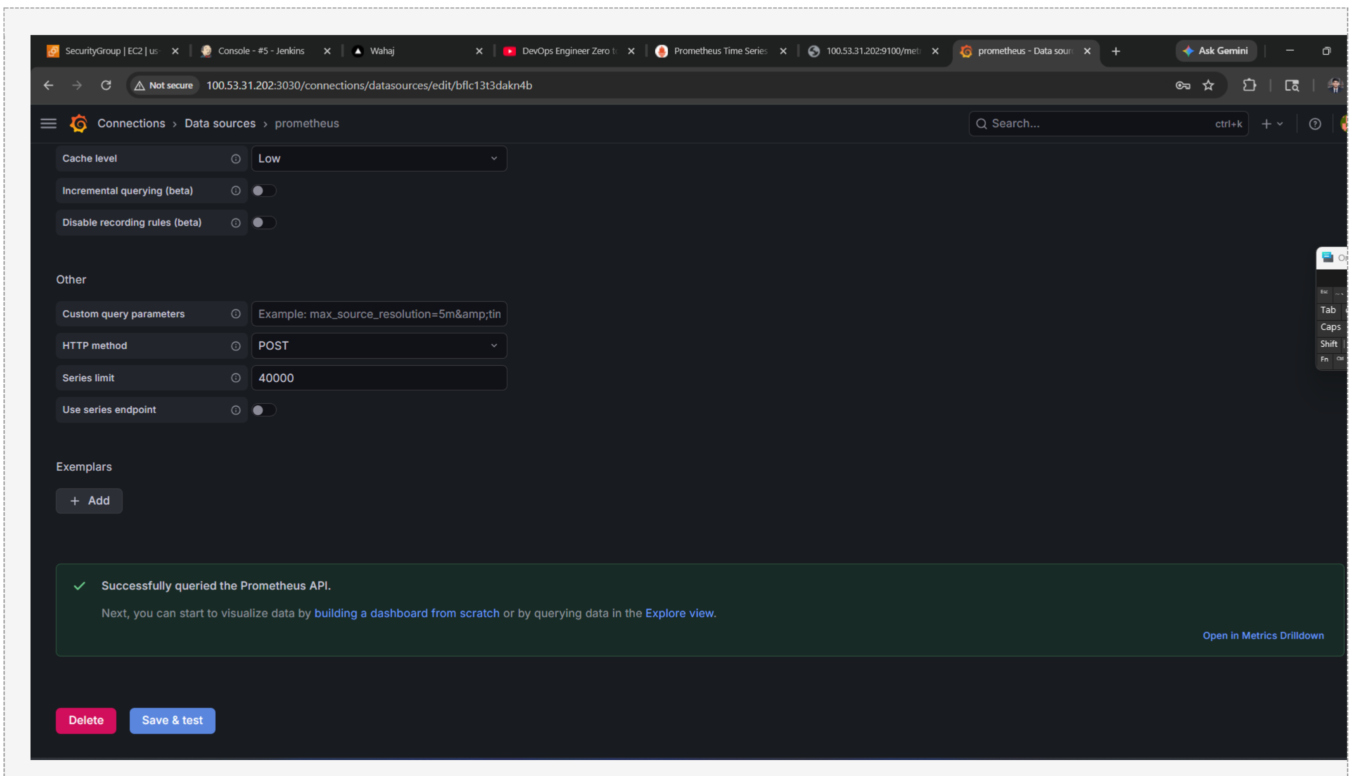
Now we connect Grafana to Prometheus so Grafana can read and visualize the metrics data.

Step 4.1 - Add Prometheus as Data Source in Grafana

1	In Grafana, click the gear icon (Configuration) in the left sidebar, then click 'Data Sources'
2	Click the blue 'Add data source' button
3	In the search box, type 'Prometheus' and click on it
4	In the HTTP section, set URL to: http://localhost:9090
5	Leave all other settings as default
6	Scroll down and click 'Save & Test'
7	You should see a green message: 'Data source is working'



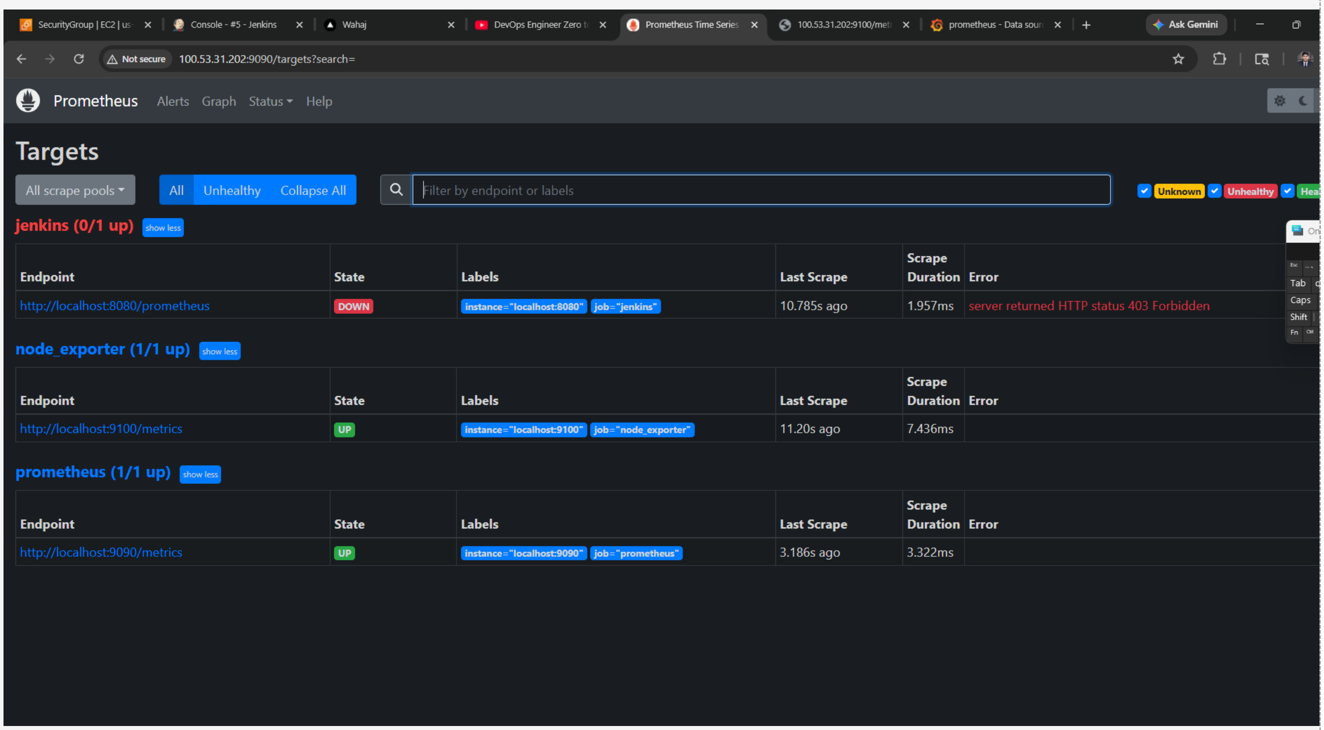




Step 4.2 - Verify Prometheus Targets are UP

Before creating dashboards, confirm that all your scrape targets are healthy:

1	Open browser and go to http://100.53.31.202:9090
2	Click on 'Status' in the top navigation menu
3	Click on 'Targets'
4	You should see all three jobs (prometheus, node_exporter, jenkins) with State = UP in green

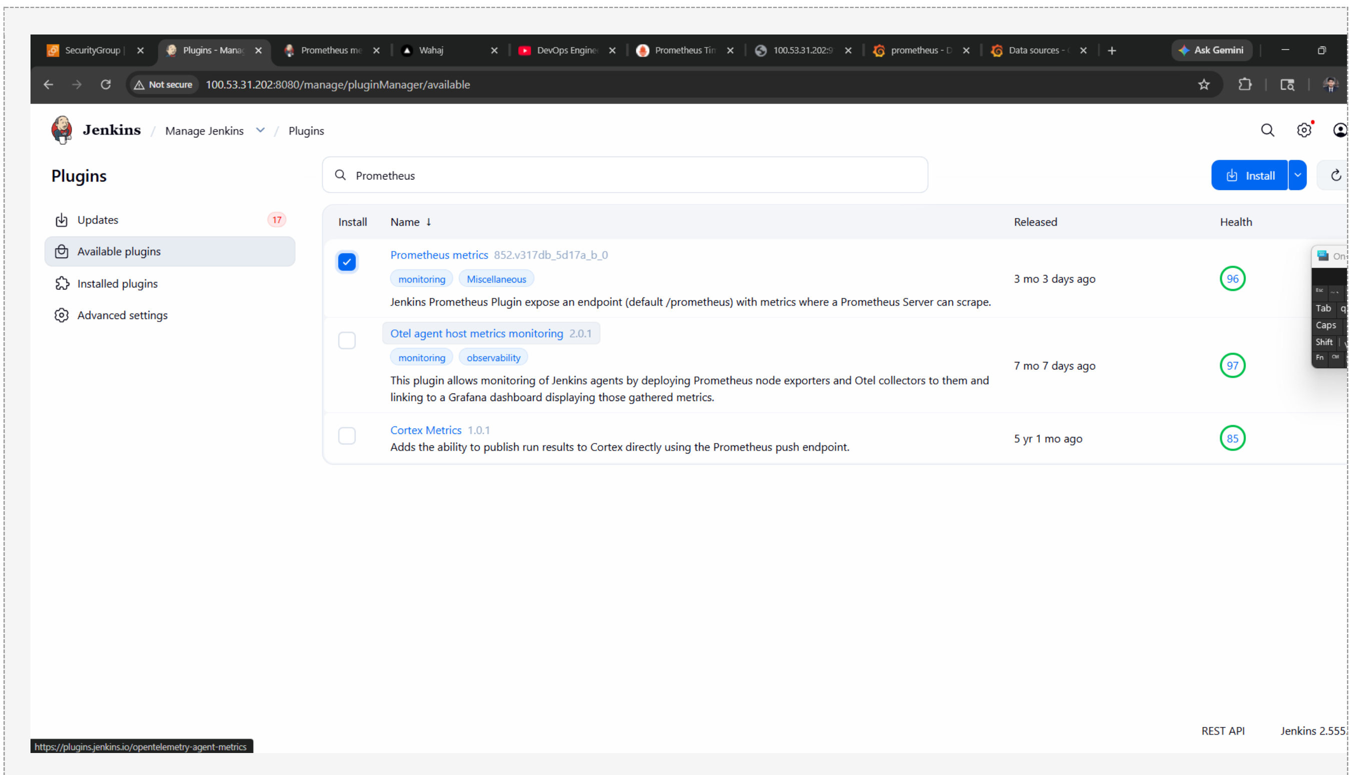


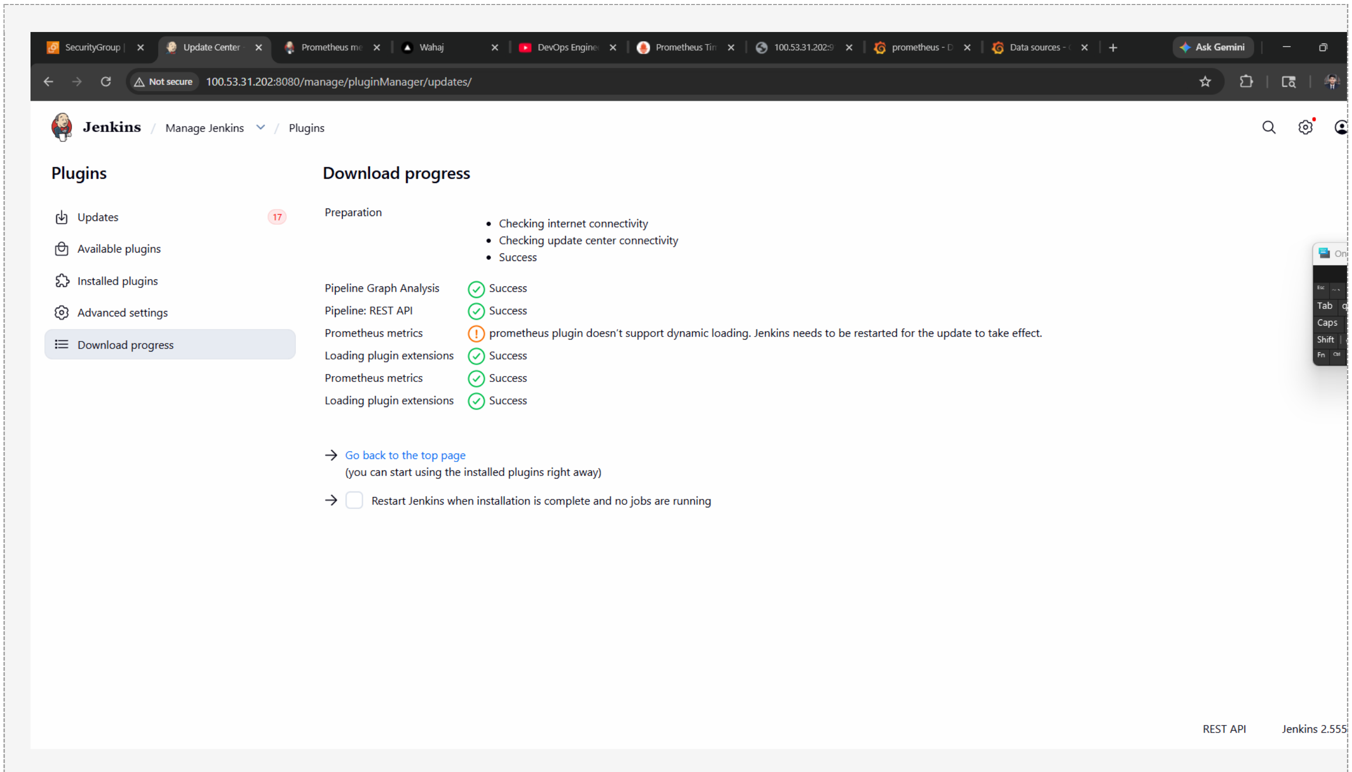
Part 5: Configure Jenkins Prometheus Plugin

To monitor Jenkins jobs and build metrics, install the Prometheus plugin in Jenkins.

Step 5.1 - Install Prometheus Plugin in Jenkins

1	Open Jenkins at http://100.53.31.202:8080/
2	Click 'Manage Jenkins' from the left sidebar
3	Click 'Manage Plugins'
4	Click the 'Available plugins' tab
5	In the search box, type 'Prometheus'
6	Check the box next to 'Prometheus metrics'
7	Click 'Install without restart'
8	Wait for installation to complete



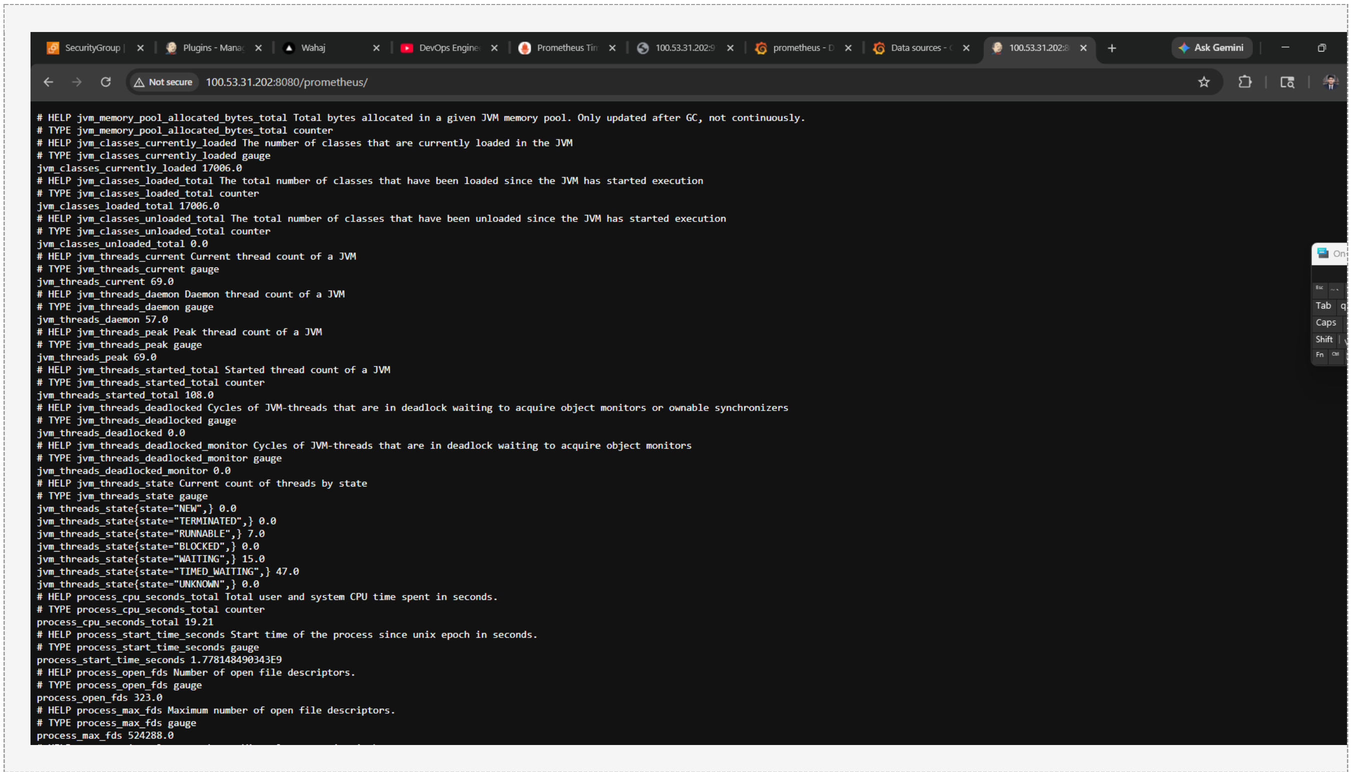


Step 5.2 - Verify Jenkins Metrics Endpoint

After installing the plugin, Jenkins will expose metrics at /prometheus. Verify it works:

```
http://100.53.31.202:8080/prometheus
```

You should see Jenkins metrics in Prometheus format (lines starting with # HELP, # TYPE, etc.)


SecurityGroup |
Plugins - Manag
Wahaj
DevOps Engine
Prometheus Tin
100.53.31.202:9
prometheus - D
Data sources -
100.53.31.202:8
Ask Gemini
Not secure
100.53.31.202:8080/prometheus/
HELP jvm_memory_pool_allocated_bytes_total Total bytes allocated in a given JVM memory pool. Only updated after GC, not continuously.
TYPE jvm_memory_pool_allocated_bytes_total counter
HELP jvm_classes_currently_loaded The number of classes that are currently loaded in the JVM
TYPE jvm_classes_currently_loaded gauge
jvm_classes_currently_loaded 17006.0
HELP jvm_classes_loaded_total The total number of classes that have been loaded since the JVM has started execution
TYPE jvm_classes_loaded_total counter
jvm_classes_loaded_total 17006.0
HELP jvm_classes_unloaded_total The total number of classes that have been unloaded since the JVM has started execution
TYPE jvm_classes_unloaded_total counter
jvm_classes_unloaded_total 0.0
HELP jvm_threads_current Current thread count of a JVM
TYPE jvm_threads_current gauge
jvm_threads_current 69.0
HELP jvm_threads_daemon Daemon thread count of a JVM
TYPE jvm_threads_daemon gauge
jvm_threads_daemon 57.0
HELP jvm_threads_peak Peak thread count of a JVM
TYPE jvm_threads_peak gauge
jvm_threads_peak 69.0
HELP jvm_threads_started_total Started thread count of a JVM
TYPE jvm_threads_started_total counter
jvm_threads_started_total 108.0
HELP jvm_threads_deadlocked Cycles of JVM-threads that are in deadlock waiting to acquire object monitors or ownable synchronizers
TYPE jvm_threads_deadlocked gauge
jvm_threads_deadlocked 0.0
HELP jvm_threads_deadlocked_monitor Cycles of JVM-threads that are in deadlock waiting to acquire object monitors
TYPE jvm_threads_deadlocked_monitor gauge
jvm_threads_deadlocked_monitor 0.0
HELP jvm_threads_state Current count of threads by state
TYPE jvm_threads_state gauge
jvm_threads_state{state="NEW",} 0.0
jvm_threads_state{state="TERMINATED",} 0.0
jvm_threads_state{state="RUNNABLE",} 7.0
jvm_threads_state{state="BLOCKED",} 0.0
jvm_threads_state{state="WAITING",} 15.0
jvm_threads_state{state="TIMED_WAITING",} 47.0
jvm_threads_state{state="UNKNOWN",} 0.0
HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.
TYPE process_cpu_seconds_total counter
process_cpu_seconds_total 19.21
HELP process_start_time_seconds Start time of the process since unix epoch in seconds.
TYPE process_start_time_seconds gauge
process_start_time_seconds 1.778148490343E9
HELP process_open_fds Number of open file descriptors.
TYPE process_open_fds gauge
process_open_fds 323.0
HELP process_max_fds Maximum number of open file descriptors.
TYPE process_max_fds gauge
process_max_fds 524288.0

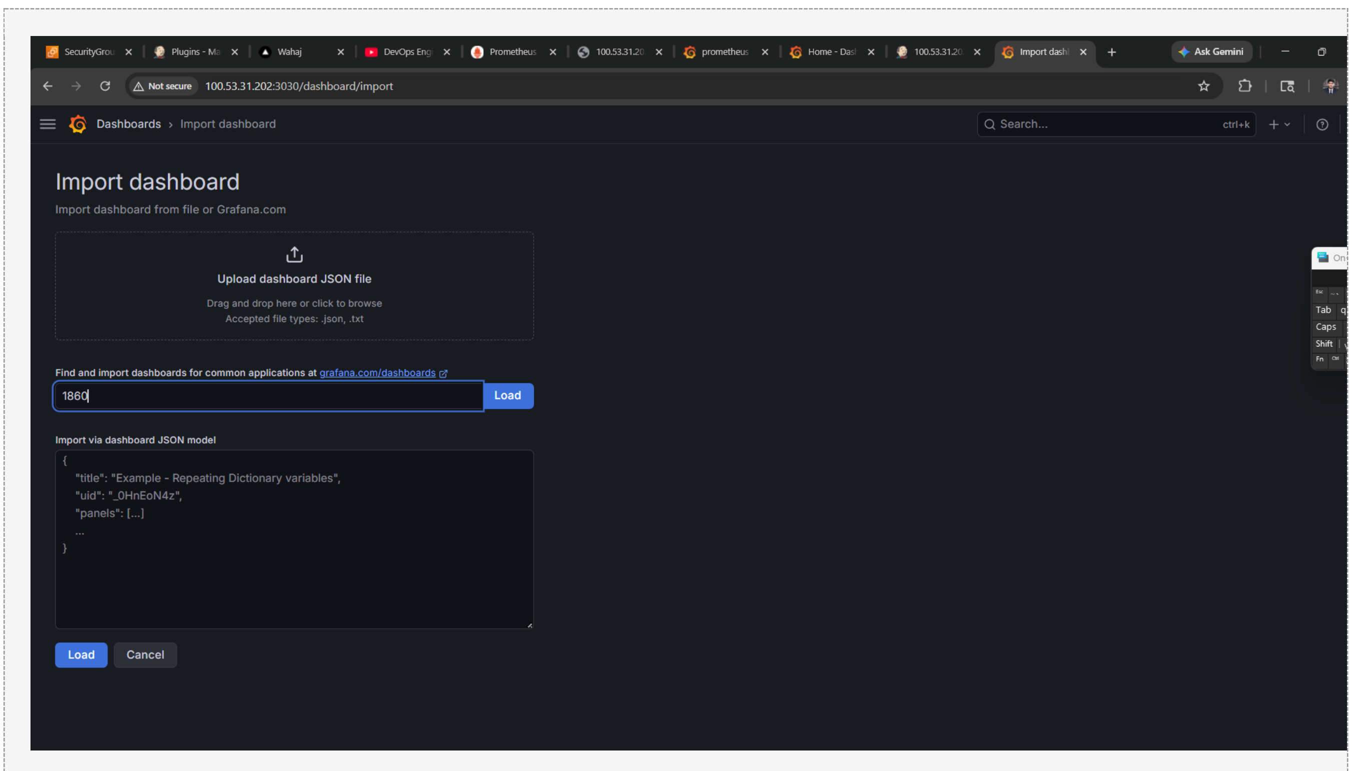
Part 6: Create and Access Dashboards

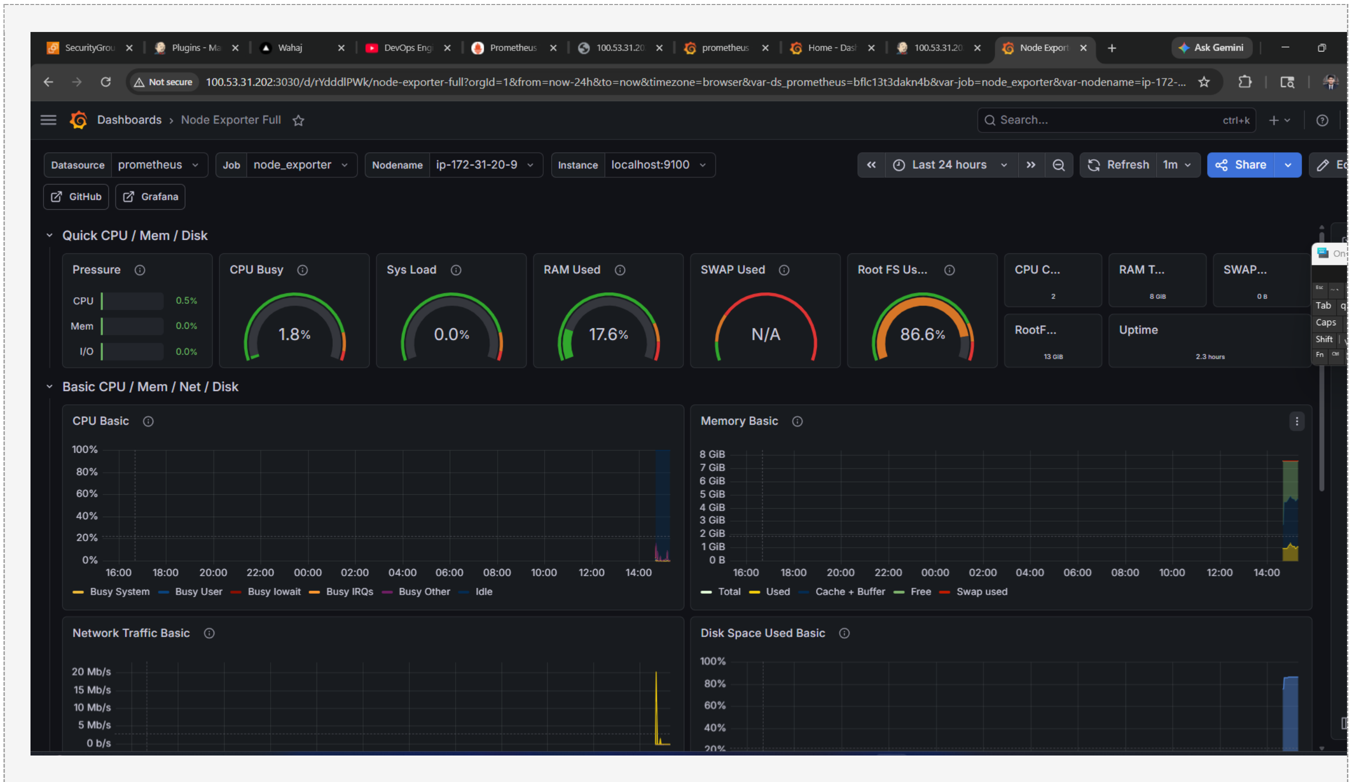
Now we set up pre-built dashboards to monitor your server and Jenkins. Grafana has a library of thousands of community dashboards.

Step 6.1 - Import Node Exporter Dashboard

This gives you a full server monitoring dashboard (CPU, RAM, Disk, Network) in seconds:

1	In Grafana, click the '+' (plus) icon in the left sidebar
2	Click 'Import'
3	In the 'Import via grafana.com' field, type: 1860
4	Click 'Load'
5	In the 'Prometheus' dropdown, select the Prometheus data source you created
6	Click 'Import'
7	Dashboard will load immediately with all server metrics!

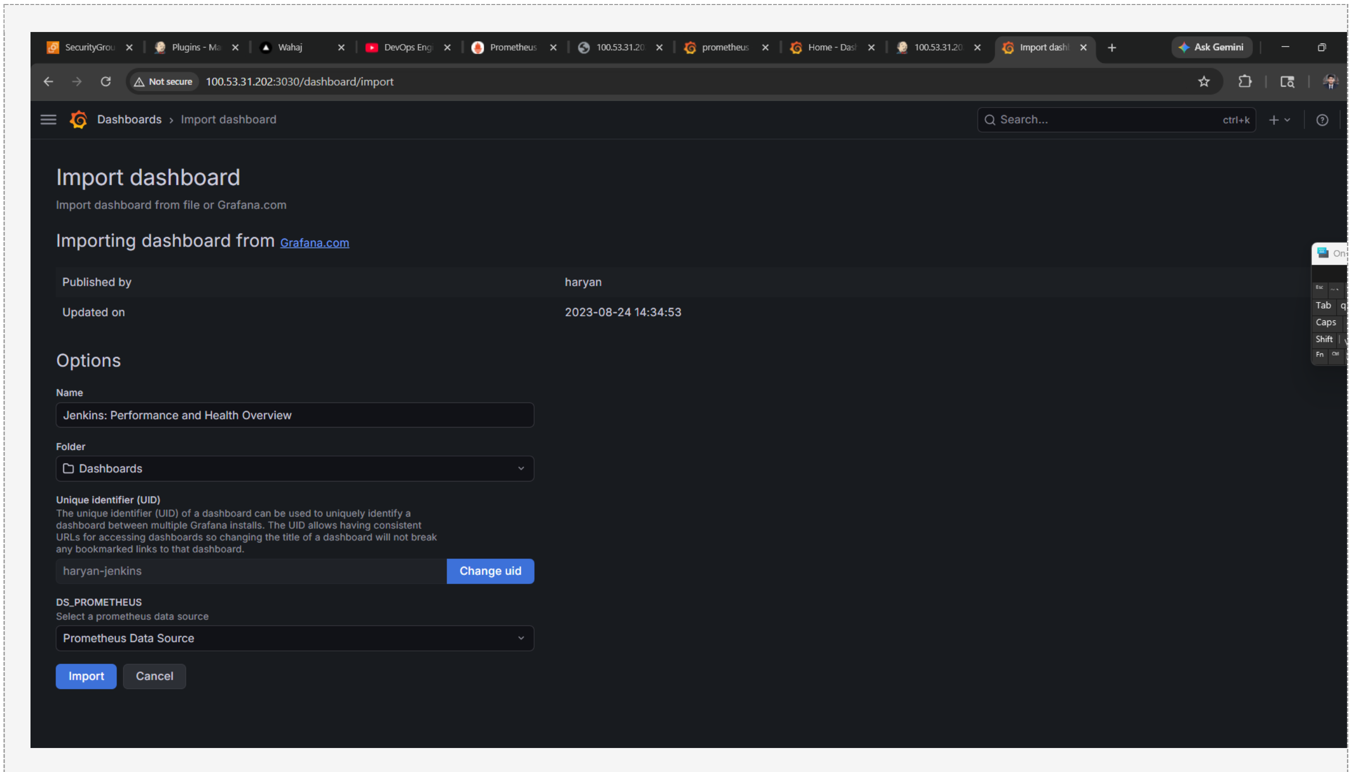


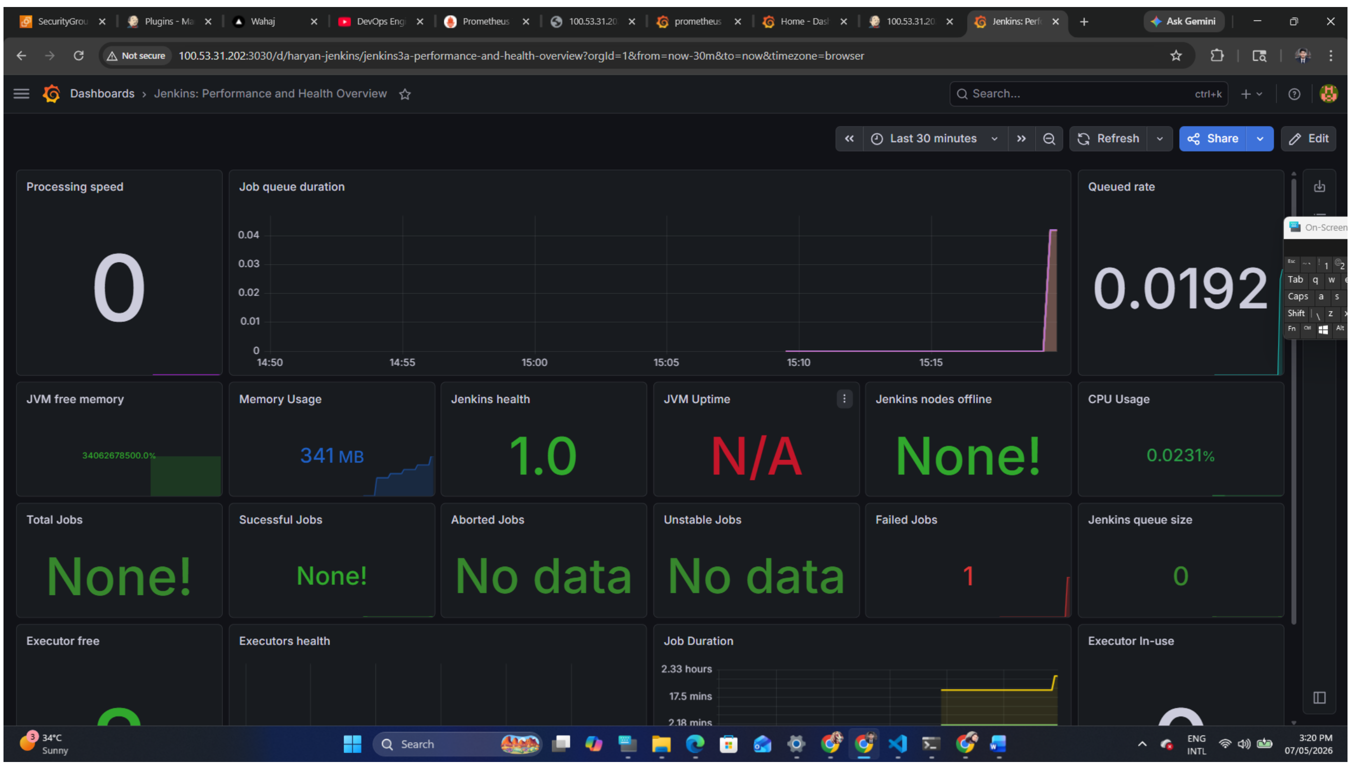


Step 6.2 - Import Jenkins Dashboard

Import a pre-built Jenkins monitoring dashboard:

1	In Grafana, click '+' icon then 'Import'
2	Enter dashboard ID: 9964
3	Click 'Load'
4	Select your Prometheus data source
5	Click 'Import'


100.53.31.202:3030/dashboard/import
Dashboards › Import dashboard
Import dashboard
Import dashboard from file or Grafana.com
Importing dashboard from Grafana.com
Published by
haryan
Updated on
2023-08-24 14:34:53
Options
Name
Jenkins: Performance and Health Overview
Folder
Dashboards
Unique identifier (UID)
The unique identifier (UID) of a dashboard can be used to uniquely identify a dashboard between multiple Grafana installs. The UID allows having consistent URLs for accessing dashboards so changing the title of a dashboard will not break any bookmarked links to that dashboard.
haryan-jenkins
Change uid
DS_PROMETHEUS
Select a prometheus data source
Prometheus Data Source
Import
Cancel


100.53.31.202:3030/d/haryan-jenkins/jenkins3a-performance-and-health-overview?orgId=1&from=now-30m&to=now&timezone=browser
Dashboards › Jenkins: Performance and Health Overview
Last 30 minutes
Refresh
Share
Edit
Processing speed
0
Job queue duration
Queued rate
0.0192
JVM free memory
Memory Usage
341 MB
Jenkins health
1.0
JVM Uptime
N/A
Jenkins nodes offline
None!
CPU Usage
0.0231%
Total Jobs
None!
Sucessful Jobs
None!
Aborted Jobs
No data
Unstable Jobs
No data
Failed Jobs
1
Jenkins queue size
0
Executor free
Executors health
Job Duration
Executor In-use
3:20 PM
07/05/2026